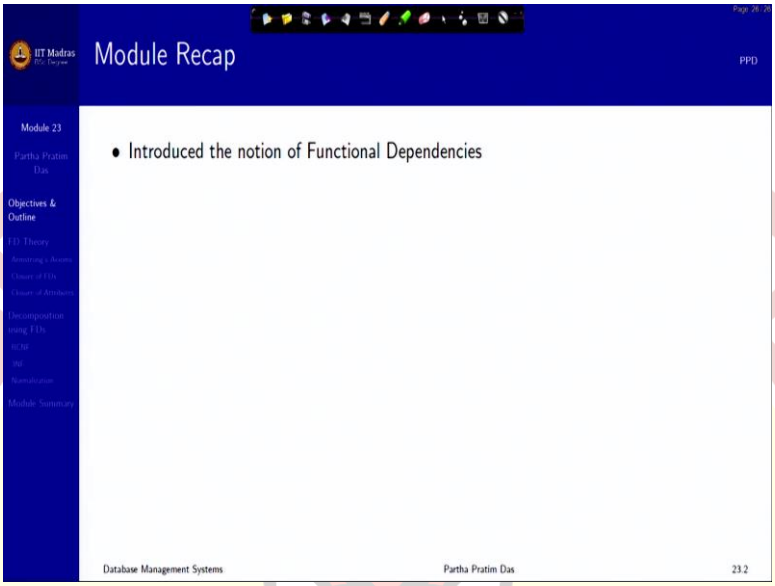
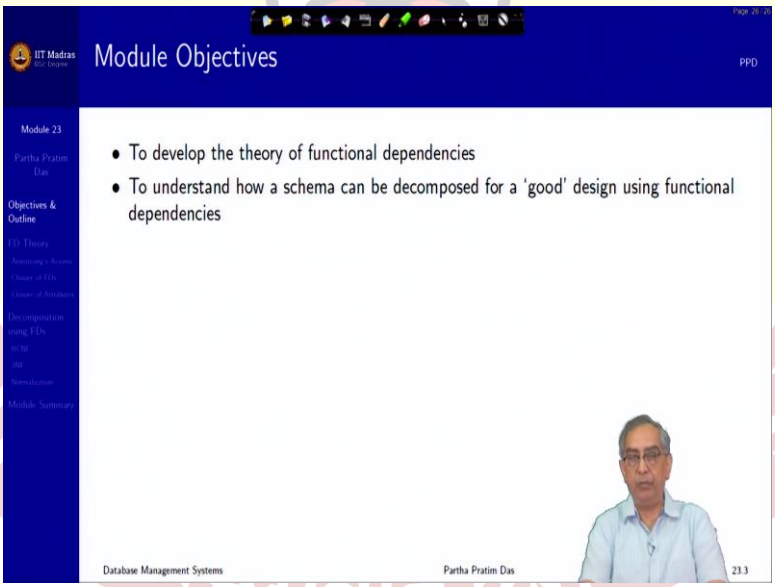
IIT Madras

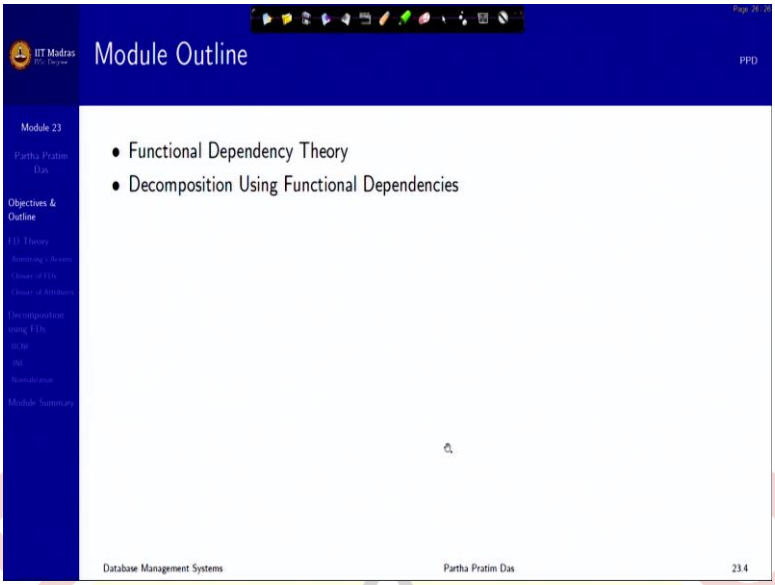
ONLINE DEGREE

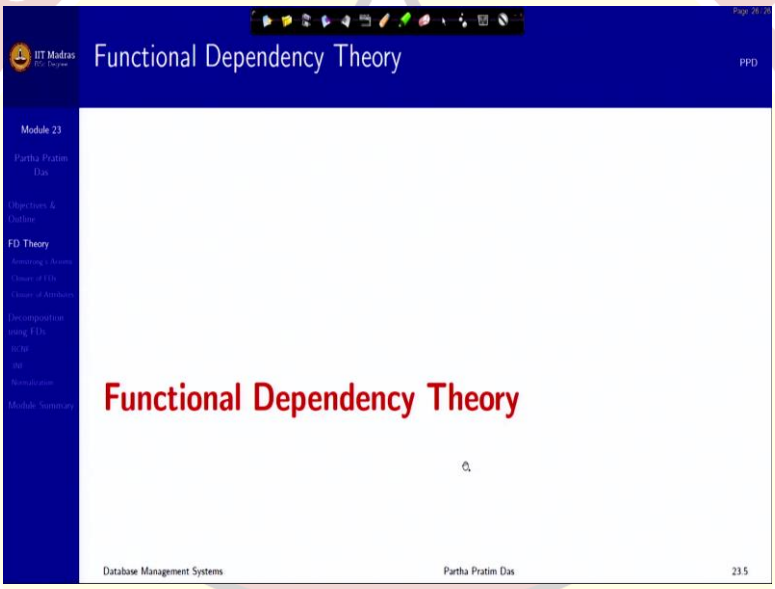
Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Relational Database Design/3

(Refer Slide Time: 0:27)



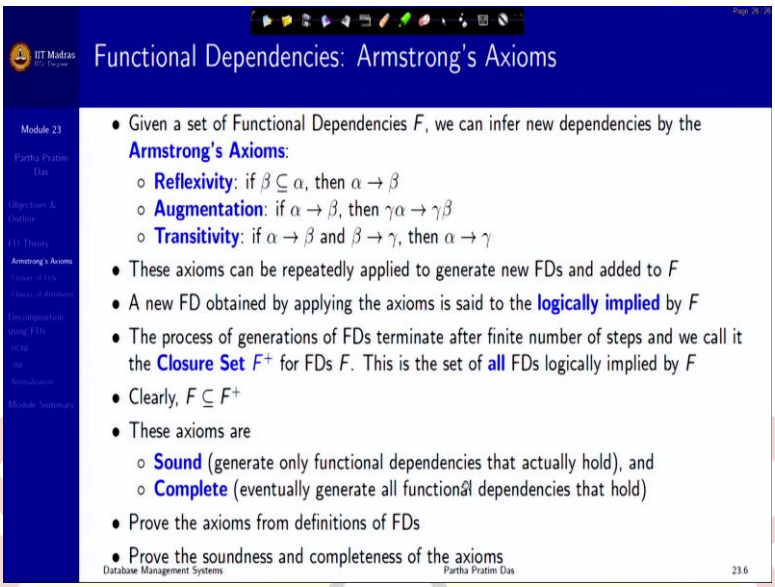






Welcome to Module 23 of Database Management Systems, in IIT Madras, online BSc programme. We are discussing design issues for relational databases. And we have introduced the notions of functional dependencies and a few basic roles, we are going to develop that into a more complete theory. And take a, take a look at how a schema can be decomposed for good design using functional dependencies and continue more on that in the next module as well. It is primarily first to the functional dependency theory.

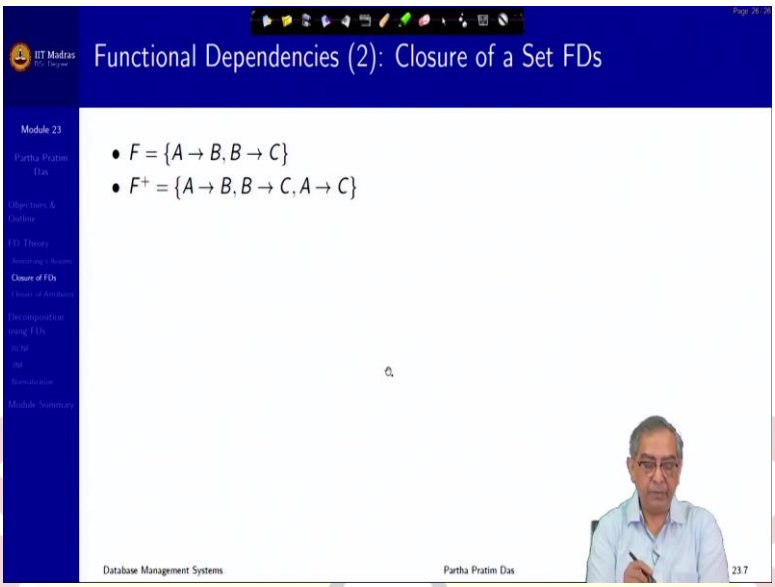
(Refer Slide Time: 1:01)



So just for a quick recap, a functional dependency F, or in the form of α determines β is a functional relationship that holds on a relational schema, so that in every instance of the schema, if we take two tuples which match on α, they must match on the β. That is the basic rule. And we saw that Armstrong's actions for that reflexivity, augmentation and transitivity. These have been discussed in the last module. So, I am just repeating for completeness here, but you must refer to the previous module for this.

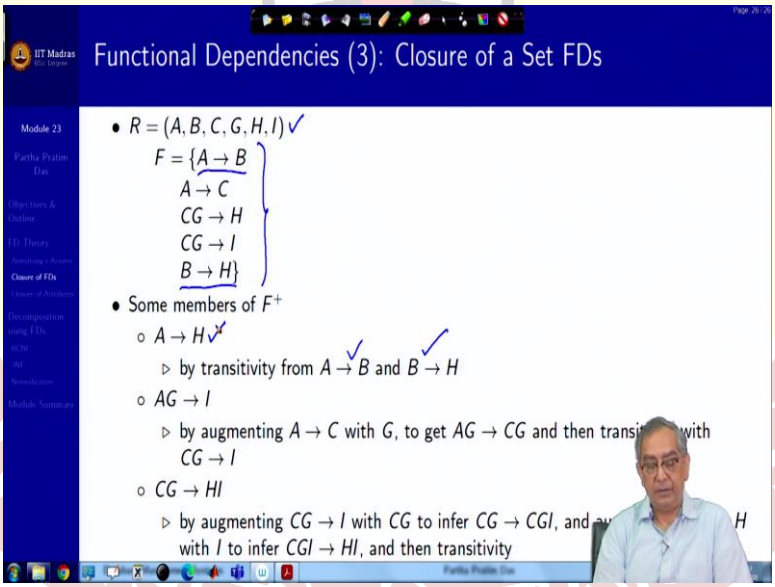
So, a new FD can be applied by applying these actions and eventually, we get a termination of that discovery of new functional dependencies and that is called the closure set F+. This we have to, these actions are sound and complete and we had, I had requested that you try to prove both the actions as well as the soundness completeness property.

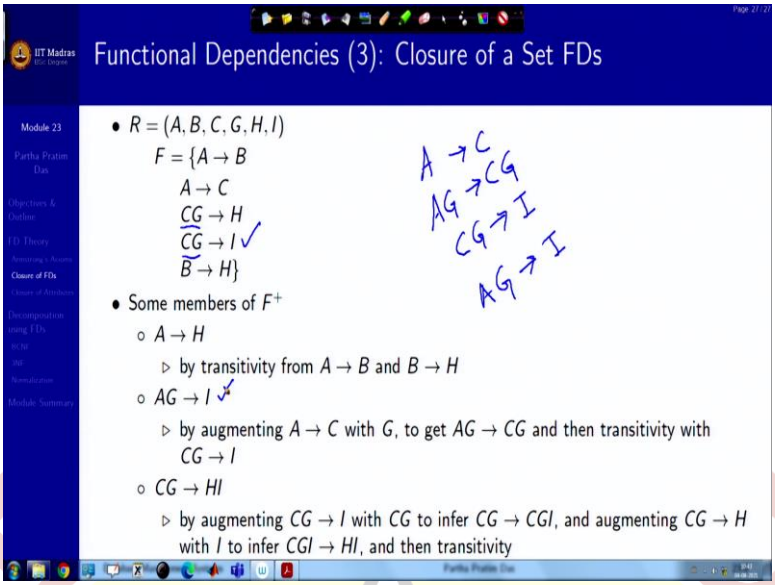
(Refer Slide Time: 2:07)

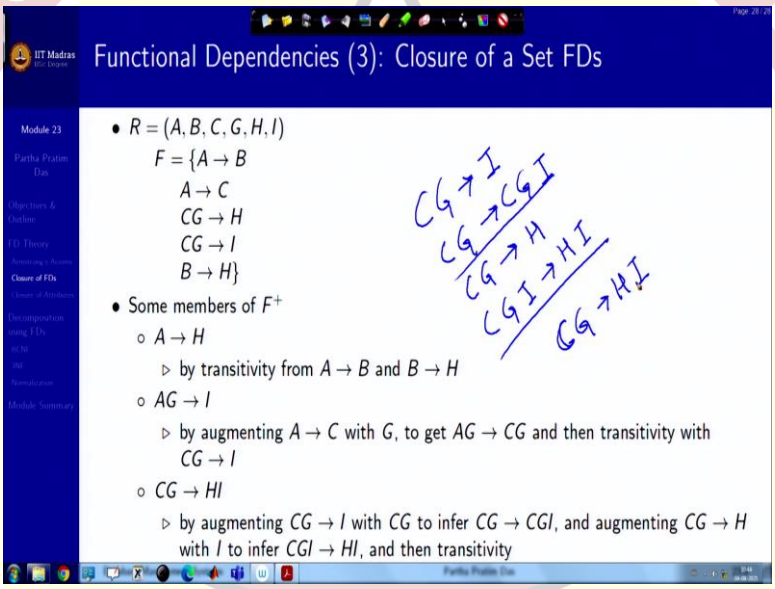


With that we saw this trivial example, A determines B and B determine C, if it is regionally given, then your closure will also have A determined C by transitivity.

(Refer Slide Time: 2:19)







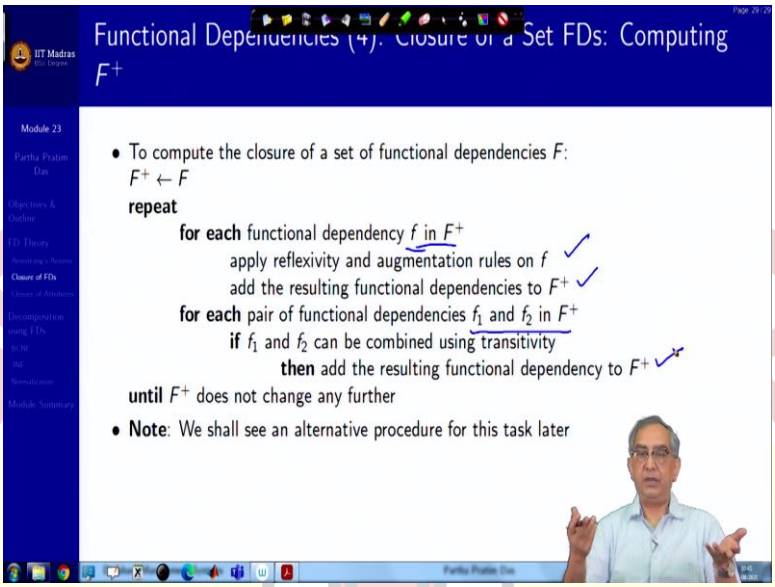
This is a little bigger example. Let us just walk through this. So, it has 6 attributes and 1, 2, 3, 5 functional dependencies. So, we can say that we have A determines B, and B determines H, these two. So, by transitivity, I will have A determines H. So, this is a new functional dependency that have discovered.

Now, let us say, A determines C. If I augment with G, the augmentation rule, then AG will determine CG. I am trying to see if I can use this rule, any of this rule. So, and then you have CG determining I. So, if you have CG determining I, so which naturally mean that by transitivity CG determining I, so by transitivity you have AG determining I. So, you have this.

Similarly, you can have CG determining HI, CG determining I you have, so let us augment it with CG, left side, right side augmented with CG. So, left side remains the same right side

becomes CGI. Then I have CG determines H, let us augmented by I. So, it will become HI. So, now, if we take these two, then we can say that CG determines HI, like this. So, you can, you can conclude more and more functional dependencies in this way, that is.

(Refer Slide Time: 4:22)

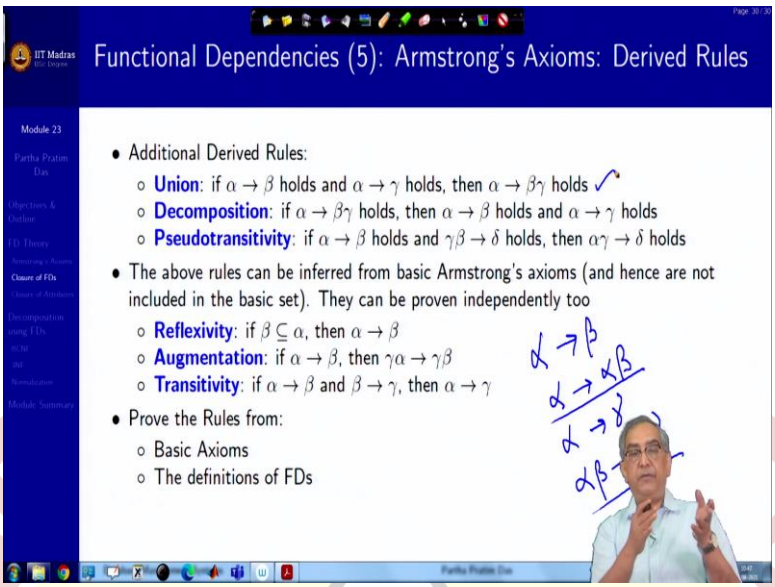


So, to do that and actually this can be done in an automated manner. We have a simple repetitive algorithm, you start by initialising the closure set F+ by F, then for each functional dependency in this temporary set F+, apply reflexivity and augmentation rules, it might generate something, add the resulting dependencies on F+, so that you have more.

And then because these are the two rules which apply on individual dependencies and the other is the transitivity which for which you take every pair in F+ and see whether they can be combined for transitivity, that is whether the right hand side of f1 is same as the left hand side of f2 and then you can apply that. If it is, then you include that, keep on doing this.

Now, naturally, since you have a finite set of attributes and a finite, therefore, a finite set of functional dependencies. So, this process is certainly going to terminate. So, if keep on doing it, when F+ is not changing any further you have got the closure set, that is a basic thing we will show, we will talk about other approaches of doing this alternative procedures also, but this is just a nave and direct approach to do this.

(Refer Slide Time: 5:50)



Now, in this process, we can we can also observe the kind of functional dependencies we were getting and also observe that we can actually enhance the Armstrong's axioms themselves. And so, these are called derived rules, because these are not rules by themselves, these follow from the original three rules.

So, the first rule is of union, that if alpha determining beta holds and alpha determining gamma holds, then alpha will determine beta gamma. That is, if two functional dependencies are the same left hand side, then the right hand side can be taken union off, it is pretty, pretty easy to to actually do this, because all that you need to, how do you, how do you prove this?

One is you can prove it from the sample, first principle the definition of functional dependencies. So, you say that if they match on alpha, they match on beta, they may match an alpha, they match on gamma naturally, so, if they match on alpha they match on beta and gamma. So, that is, that is one way.

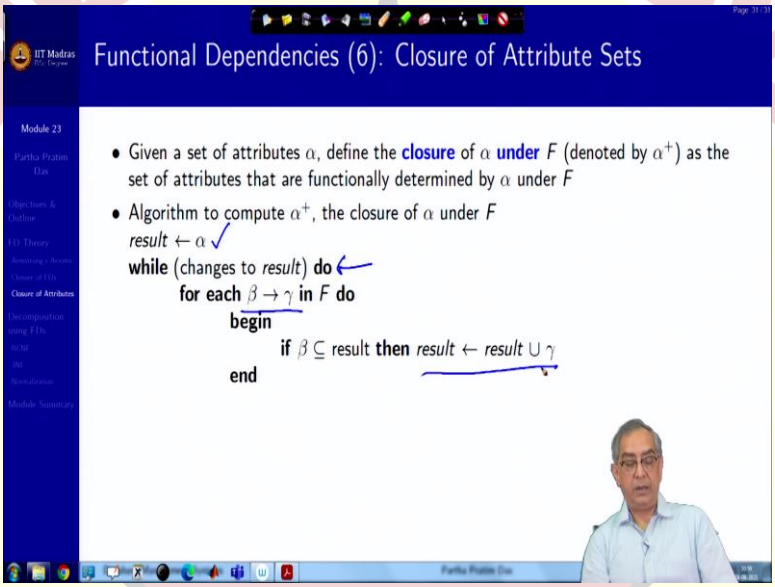
Can we use the rules, that is what you will have to see what you can do? For example, say if I, if I augment the first rule, then I will get I have this, suppose I augment the first rule with alpha. So, I will get alpha determines alpha beta, I have alpha determining gamma, I will augment this rule by beta, then I use transitivity between them, so, I will get this.

So, these are the kinds of things we were doing in the example. So, this can be formalised in terms of rules then similarly, decomposition is kind of the reverse of the union that if alpha determines beta gamma, then alpha determines beta and alpha determines gamma. And

pseudo transitivity is alpha determines beta and gamma beta determines delta, then gamma alpha will determine delta. Again, in the same way you can augment and do this.

So, try to, this is so, these rules are not included because they are not required for the completeness, these rules are still sound. And since they follow from these rules, so, they are not required for the completeness. The completeness, but you need these three basic rules, reflexivity, augmentation and transitivity for the completeness. So, you can prove these derived rules from the basic actions. And also, I am using the three basic rules or from the definition of functional dependencies itself.

(Refer Slide Time: 8:50)



Now, we will now introduce another concept. So, this was closure set of functional dependencies. Now, we will talk about the closure set of a, set of attributes. Suppose, you have a set of attributes alpha, then under the set of functional dependencies F, we compute the closure of α, as α+.

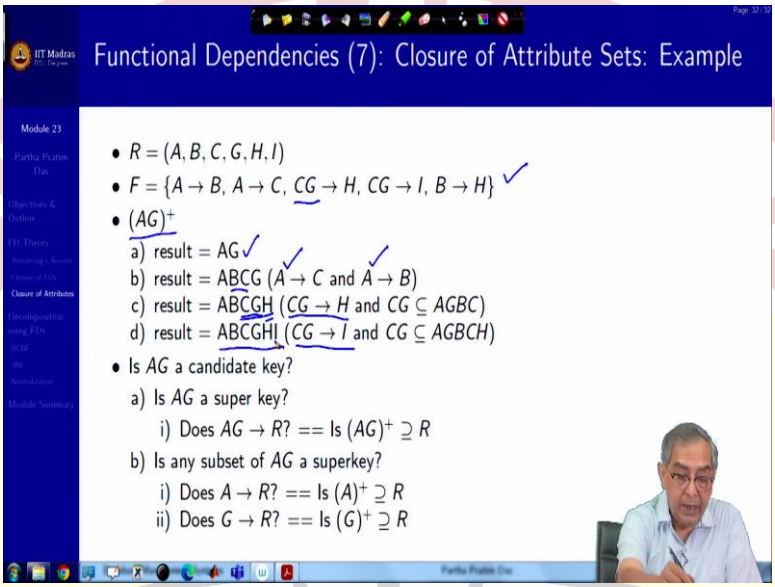
So, what is α+? α+ is a set of attributes which can be functionally determined by α and F. So, it should be possible to functionally determine each and every attribute of α+ from alpha applying the functional dependencies that exist in the set F. So, any, for any set of attributes the closure set is necessarily with respect to the set of functional dependencies.

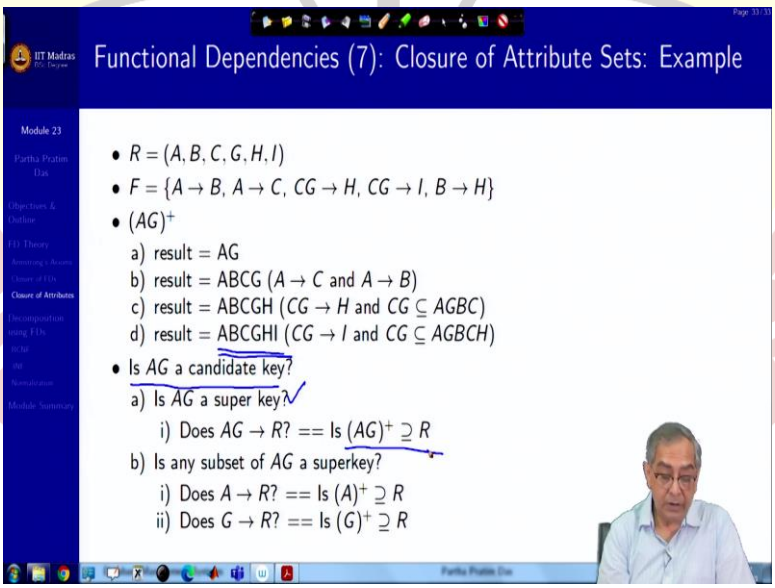
So, the way to compute that would be trivial, you initialise the result with the set alpha because obviously alpha will be a subset of α+ and then track if your result is changing. So, if we have beta determining gamma in F, then if beta is already included in the result that

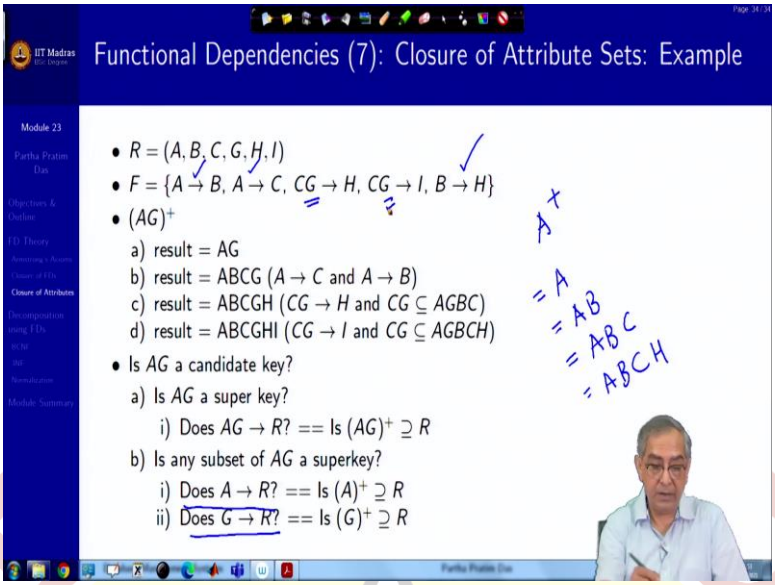
means alpha determines this set of attributes. So, if beta is a subset of that, then alpha determines beta. So, and beta determines gamma.

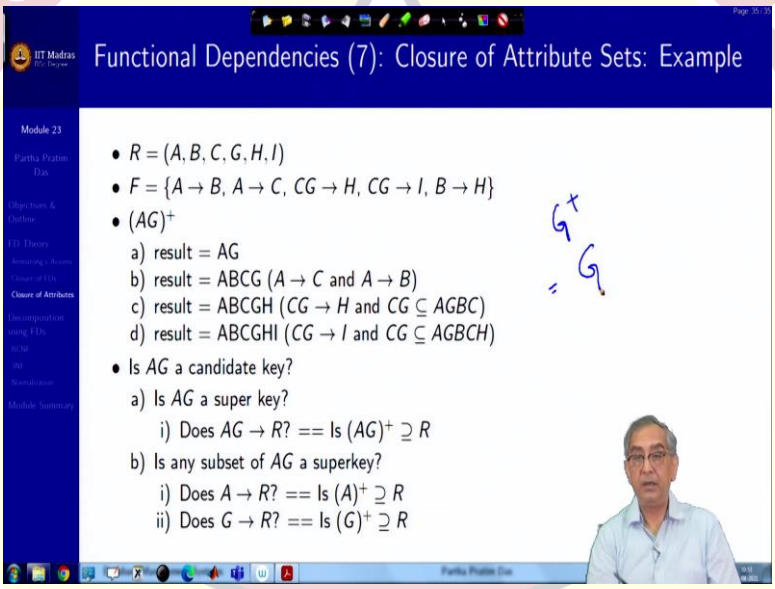
So, by transitivity alpha will determine gamma. So, you include gamma in the result and keep on doing, till changes happen, very straightforward algorithm but very useful algorithm. So, just a quick look of the same set.

(Refer Slide Time: 10:39)









So, let us say we are computing the closure of AG. So, initially result is AG, then we have A determines C, we have A determines B, A is included. So, B and C will get included one by one. Now, we have CG, CG is included here. So, CG determines H gets included. And we have G, we have H added to it, this CG was already included. So, CG determines H. So, H gets in.

And finally, CG determines I. So, in the same way, I also gets included. So, this is your closure of the set AG, attributes AG with respect to the functional dependencies. Now, the question, for example, a simple question is, is AG a candidate key? To answer that, two things, we will have to see, one is it is a super key and the other is it is minimal.

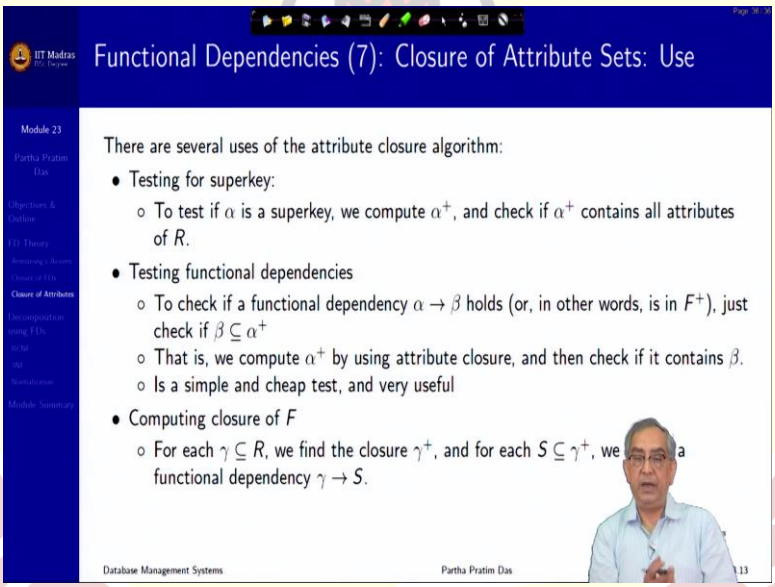
So, it is a super key. Why is it a super key? Because if you see (AG)+ then you have all the attributes of R. So, you have all the attributes of R. So, therefore, it is functionally determined

determining each and every attribute. So, it is a super key. For checking minimality, what is to see, that the subsets are not super keys.

So, does A functionally determine R or G functionally determine R? You have to check. And, it is easy to see if you see A+, what will that mean? We will start with A, then we will have AB, then we will have ABC, by these two, have ABC, so you will have ABCH. And that is where you stop, because quite, quite understandably.

Now, you can, in this example, you can very well see that G actually does not belong to the right hand side of any functional dependencies, which means you can never have a key which does not have G. So, this part is obvious. How about G+? What if we try to do G +? G+ is, G alone does not feature anywhere. So, G+ is. So, neither A+ nor G+ cover all the attributes of R. So AG is a super key and is minimal. So, it is a candidate key.

(Refer Slide Time: 13:47)

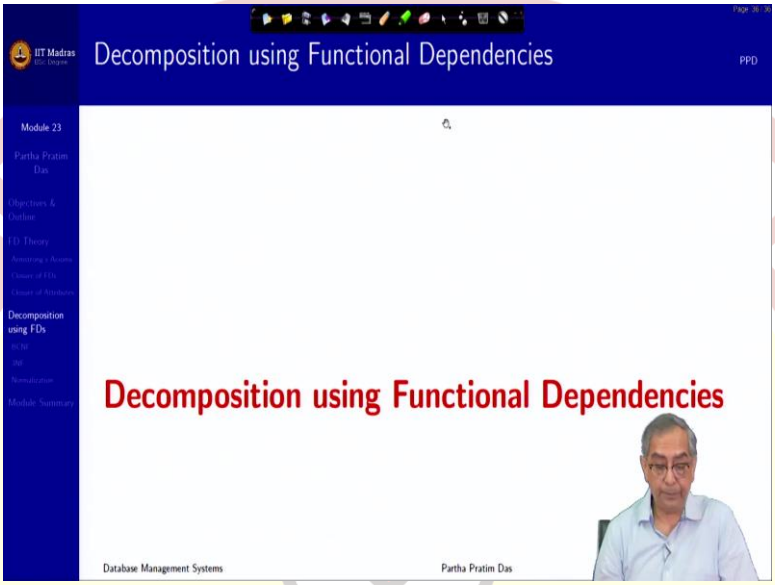


So, in this way, the the effectiveness of this algorithm is, it can help you solve related problems that we often need to solve. For example, what we just saw is testing for a super key. If you want to test, if a functional dependency holds on a relation or not, you are just given a functional dependency, alpha determines beta, does it hold? Is it a, is it an element of F+ you have to find out?

You can do that, but it is more expensive process. But all that you need to do is to check if α+ includes β. If α+ the closure of alpha includes β, then α will determine β. So, you just need to compute α+ and see if β is a subset. So, it is a simple simple check quick and useful.

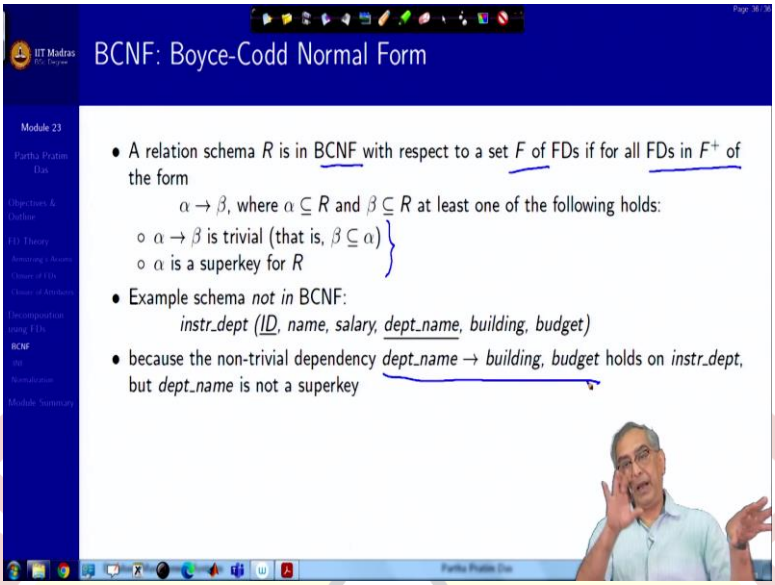
You can use this closure of attributes also to compute the closure of F. Because for each gamma in R we find the closure of γ+, and for each S in γ+ that is a closure, you compute the functional dependency gamma to S, just convince yourself that by this process you will actually get it, get the closure of F and it is more efficient. And we had mentioned that we will have alternate ways of computing F. So, this is F+, this is one with that.

(Refer Slide Time: 15:23)



Now, this is this we are not completely done with the theory of functional dependencies, but we have given the key concepts of functional dependency how to get more and more functionally determining set through Armstrong's axioms and particularly, how to compute the closure of functional dependencies and most importantly the closure of sets of attributes.

(Refer Slide Time: 15:49)



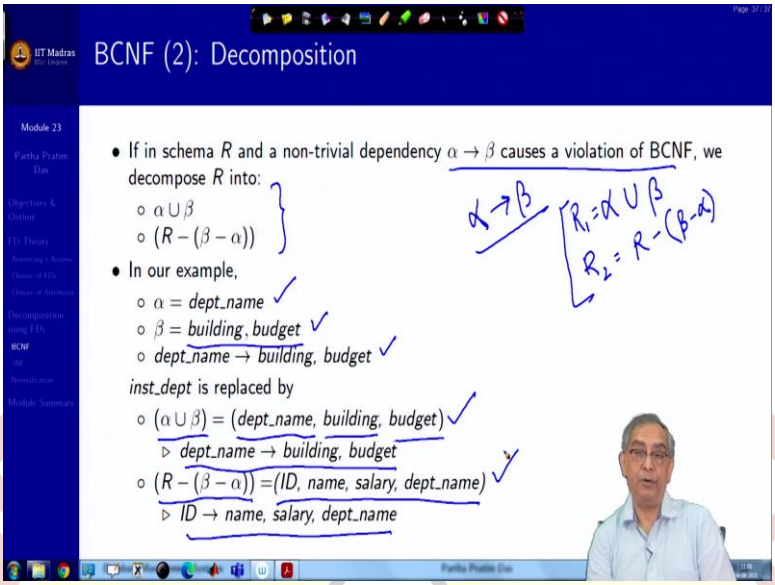
So, we just take a quick look we will have more formal stuff on this in later modules, but just a quick look of how functional dependency helps in doing a decomposition. We have seen decomposition, we have known why decomposition is important, but all that had been add on.

So, what do you say that there is a normal form, like what seen first normal form, one NF you have seen, similarly, another normal form is known as Boyce Codd Normal Form, BCNF. Boyce-Codd at the, at the scientists who actually designed this to normal form. So, we say that a relational schema R is in Boyce Codd Normal Form obviously, R needs to be in first normal form, that is always there.

It is in Boyce Codd Normal Form with respect to a set of functional dependencies. If all FDs, if all functional dependencies in the closure has satisfied any one of these properties. What are the properties? Every functional dependency is either trivial means right hand side is a subset of the left hand side, it is trivially, reflexivity or the left hand side is the super key for R for the relation.

So, if you look at our again, instructor with department relationship, you will see that this is not in Boyce Codd Normal Form, because I have a dependency here which is department name deciding building and budget where neither it is trivial nor department name is a key of the relation. But if you think of the instructor relation, and if you think of the department relation, you will find that each one of them is in Boyce Codd Normal Form.

(Refer Slide Time: 18:01)



So, we just do this little workout here. So, how do we, so this the previous condition tells us that if every alpha determines beta is either trivial or the left hand side alpha is a super key, then it is in BCNF. So, what if some alpha does not satisfy this, if some alpha violates this, then what do you need to do? You need to decompose.

How you decompose? You make one relation which is alpha union beta and you make the other schema as R – (β - α). That is from this violating functional dependency take the right hand side, take out the trivial part, trivial part is the attributes which are also in alpha, take out that. So, it is it is a unique part that remains and take that out from set from the whole set of attributes R and you divide it in this way.

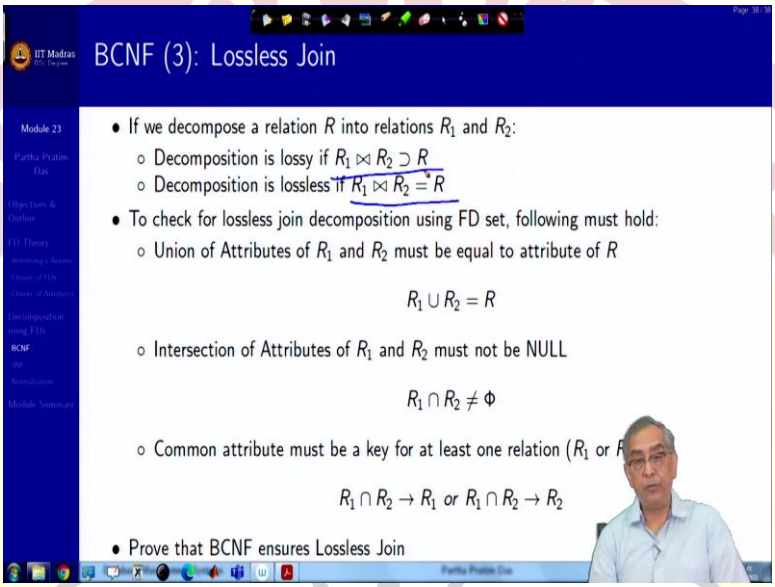
We will understand why suddenly this formulation has come they have done a lot of mathematics and come to that, we will, we will in a simple logic understand this over a period of time. So, in our case, alpha is department name. Beta is building budget, because we have this dependency, functional dependency which is violating the BCNF.

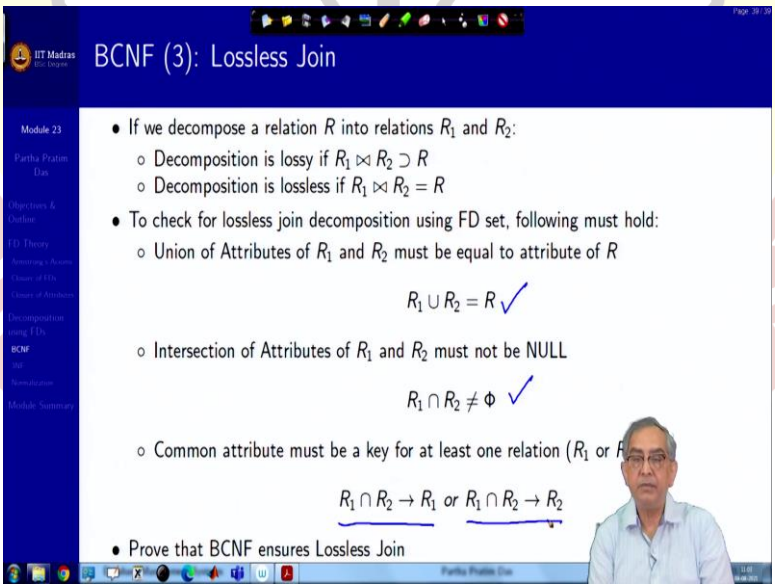
So, what you will do, you will have one which is alpha union beta that is department name, building budget and the other is R – (β - α). What is (β - α)? β - α is building budget only, because department name is not there on the right hand side is already unique. So, building budget has to be taken out from R. So, from R which is the institute, instructor with department relation we had.

So, if you take out building and budget, what you are left with is our original instructor relation. So, where ID functionally determines name, salary and department name. So, and in

this department, name functionally determines building and budget. So, both of these relations are in BCNF by the second rule, because every dependency that holds every non-trivial dependency that holds, has a key on the left hand side. So, what we saw actually is a BCNF form of normalised relationship of the instructor and department. Now, we are coming to that through this formal steps. I hope that is clear. We will have workout examples later on and so on.

(Refer Slide Time: 21:14)





Now, the question is again we have decomposed. So, the question is, is it, are we going to retain the information? Are we going to recover all the information? So, is this a lossless decomposition or a lossy decomposition? So, just to recap, this is your lossless decomposition, when you have decomposed R into R1 R2 take them as instances and you

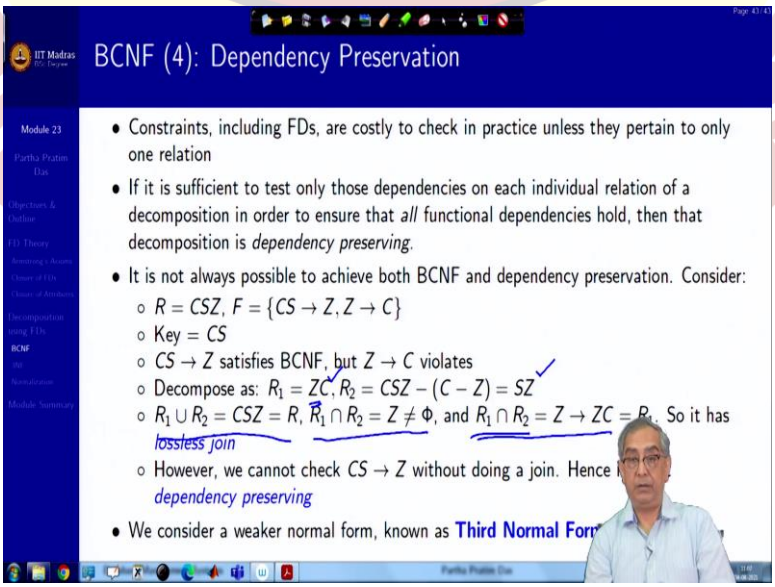
make a natural join, you must get back R in every possible case. If it is not, if you get something more than it is the lossy one.

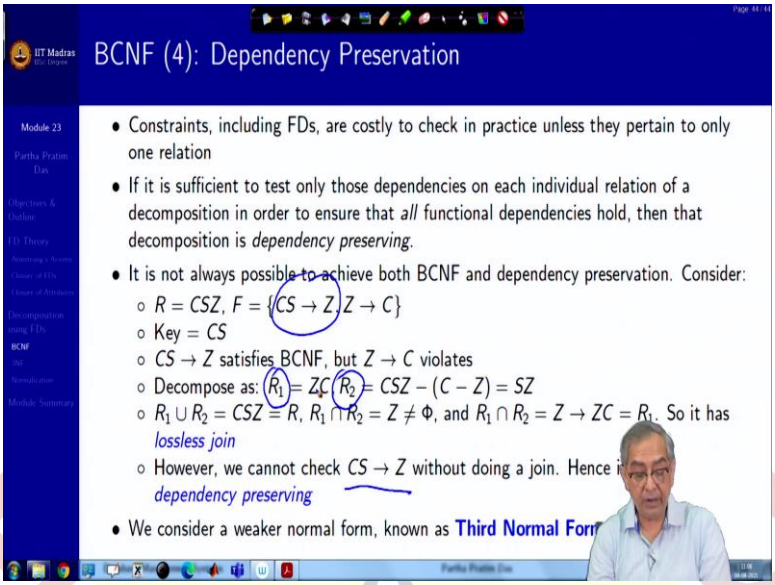
Now, these are just you know, you know statements, I am making, we will, we are not proving this here. So, for a lossless join decomposition, the following must hold on, one is, what does should hold? That one is the union has to cover the whole attributes, there must be at least one common attribute to do the joint and this is important part, that the common attributes are R1 intersection R2 must be a key either in R1 or in R2 or in both.

So, one of them must be, if that happens, then we will have lossless join, then you will always be able to recombine and do that, it is not difficult to, first two conditions are obviously trivial. The last one is not difficult to see because if one is key, then that will hold the records and from the other relation, we will be able to naturally add them like between instructor and your instructor and department the the the intersection set if we look at, that is the instructor, the intersection set is department name, R1 intersection R2 is department name and department name is the key here.

So, when you do the natural join from this relation department, you are getting unique entities to get related. So, there is no chance of getting more combinations than you originally had. Because you will be able to pick up only unique one. So, if that happens on either of this, then because natural join actually is symmetric operation. Good if you understand, do not worry if you have not understanding this reasoning, you will slowly understand that.

(Refer Slide Time: 24:16)





Now the second question is does it preserve all the dependencies? Which means, what it means is can I check every dependency that holds, can I check it in a single relation or I will have to make join, make a bigger relation and check if the dependency holds. Like in the, in the case of department and instructor, we saw that all functional dependencies are involving either the attributes of R1 or the attributes of R2, so each one of them can be checked in the respective relation.

It happens to preserve the dependency, because the reason it is important is often it is very expensive to compute the natural join, because it needs to do the Cartesian problem. So, if to ensure that the dependencies are preserved, if I have to, the system has to compute natural join, then that design is going to be a very bad one. So, the department, the system would not want those kinds of, so, we would want that after decomposition also, it should be possible to check every dependency on one or more of the individual relations only.

So, let us say, let us see does BCNF give us that? It gives us the lossless joint information preserving. So, does it preserve the dependencies? So, let us take an example. So, we have three attributes and we have CSZ actually means city, street, zip code, what we call PIN code. So, CS determines that, city and street determines as a put, and zip code determines the city, these are the functional dependencies obviously, CS is the key.

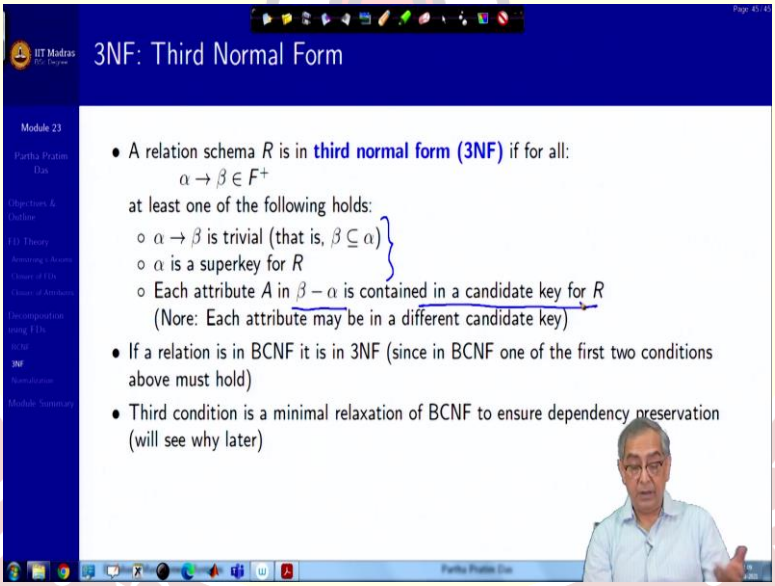
Now, CS determines Z is in BCNF, but Z determines C violates because it is neither trivial nor Z is a key. So, what you will have to do, you will have to go for a decomposition based on the Z determine C. So, one is Z union C, which is ZC, and another is CSZ – (C - Z) that beta minus alpha, which gives us SZ. So, we have two relations ZC and SZ. Then union covers everything, first condition satisfies intersection is not null, it is Z and R1 intersection

R2 is Z, which by augmentation, determines ZC, Z determines C, so, Z determines ZC. So, what is ZC, ZC is R1. So R1 intersection R2 functionally determines R1, so this is a key.

So, with these three conditions satisfied, we can see is really, we knew that this will happen because it is a property of the BCNF. But in this example, also we have through example, shown that this is actually BCNF, actually having lossless join. However, if you think about CS determining Z, this functional dependency, you cannot check it in R1, neither can you check it R2, because it involves all the three attributes.

So, to be, to check it, you will have to make a join of R1 and R2 get back to the original big schema, and only then you will be able to check this dependency. Hence, this is not information preserving. So, obviously, the validation of lossless join property can be proven, this cannot be proven, because it does not hold. So, we have given a counter example to show that it does not hold.

(Refer Slide Time: 28:45)



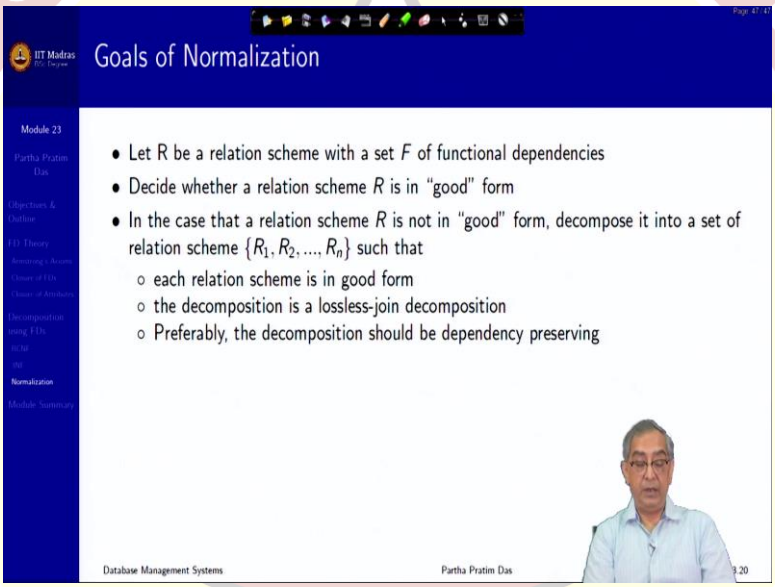
So, there is a, we can normal form known as third normal form, which does decomposition with both of these properties. It is, do not worry we will again have examples of each of these. So, it has three condition, the first two are the same as BCNF, that if a dependency alpha determines beta is in the closure of F, then either alpha determines beta is trivial or alpha is a super key.

But it has a third condition, which relaxes from the BCNF. So, it says each attribute A in β - α, the non-trivial part is contained in a candidate key for R. And the interesting thing is it does not have to be contained in in the same candidate key. The beta minus A may have three

attributes. They may be contained in three different candidate keys, two different candidate keys, one candidate key, as long as they are a part of some candidate key, we will say that this relation is in third normal form.

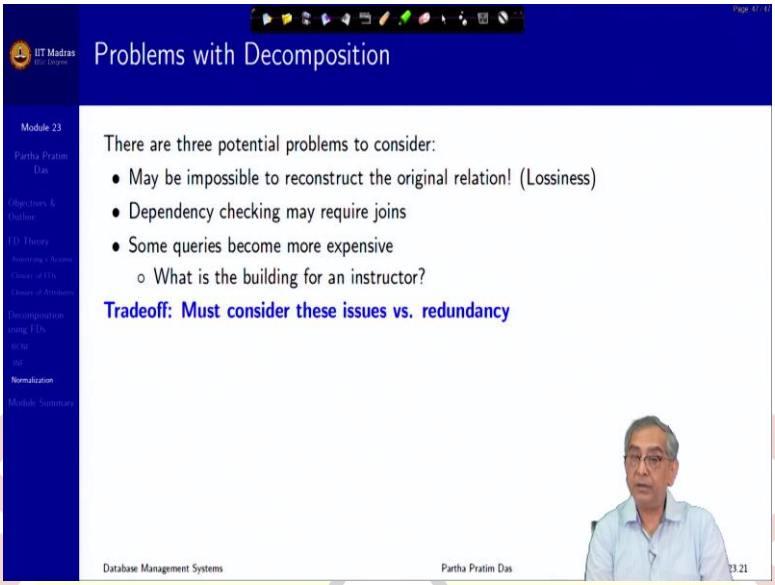
Obviously, if a relation is in Boyce Codd Normal Form it satisfies the first two and therefore, it is in third normal form, but not the other way around. But, this third condition is a minimal relaxation of Boyce Codd Normal Form to ensure dependency preservation. So, that is, we will we will come back to this, this again, I just wanted to give you the glimpses slowly of kind of things that we are developing into, we will come back to third normal form and discuss more about that in a later module.

(Refer Slide Time: 30:26)



So, the goals of normalisation are now should be quite clear that we want to make a good relational schema design. And if the relational schema is not good, we will decompose into n relational schemas as required. So, that each schema is in good form, the decomposition is lossless join and preferably it should also be dependency preserving key areas.

(Refer Slide Time: 31:04)

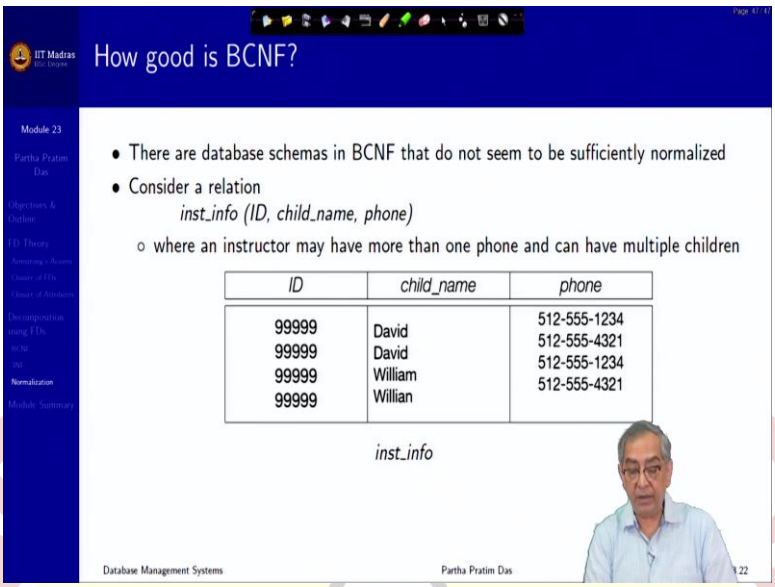


So, you need this because otherwise you will have problems with the decomposition because there are three potential problems, one is it may be impossible to reconstruct the original relation for which we need lossless join. Dependency checking may require joins which is very expensive. So, you need dependency preservation and also some queries may become more expensive as you decompose. This is, this is even keeping a certain normal form, there may be multiple choices that come to you. And then you will have to consider these kinds of factors.

For example, I want to know the building for an instructor. After the BCNF forms have been done, that decomposition has been done from the, from one table have to get the department and go to the department table and get that or do a join. I mean, it is it is more expensive than the non-normalised. So, there is always a question of trade-off between these issues.

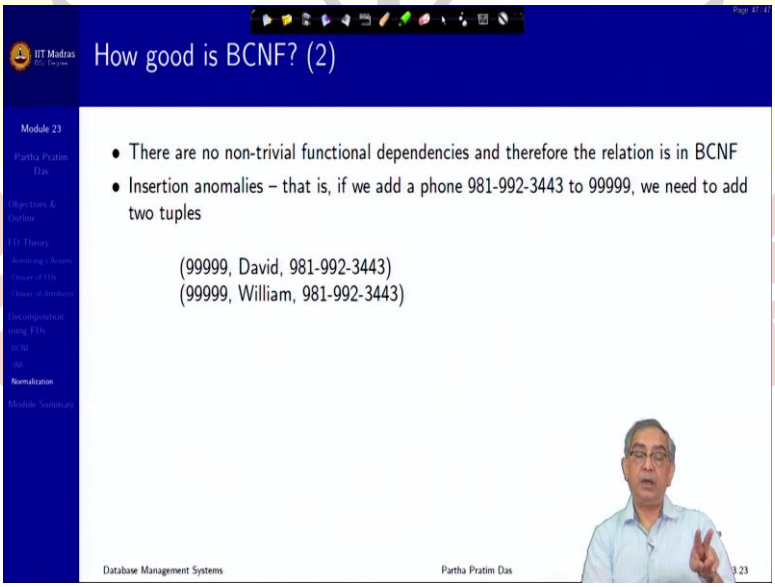
That decomposition, all I am trying to point out that there is a design goal to decompose and make things good. At the same time, as I said that the goals at times are contradictory. So, we have to make a balance between these issues and the consequences if you decompose last less you have more redundancy anomaly, so what hurts you more that we will have to see.

(Refer Slide Time: 32:32)



Some of the other aspects of BCNF let us say there is a database schema, as a, as a database normalisation, BCNF is a very strong normalisation, but yet it is not sufficiently normalised. So, we are considering a instructor info table, which is ID child name and phone and every instructor may have multiple children and may have multiple phones. So, this is an instance which you can see is full of redundancy.

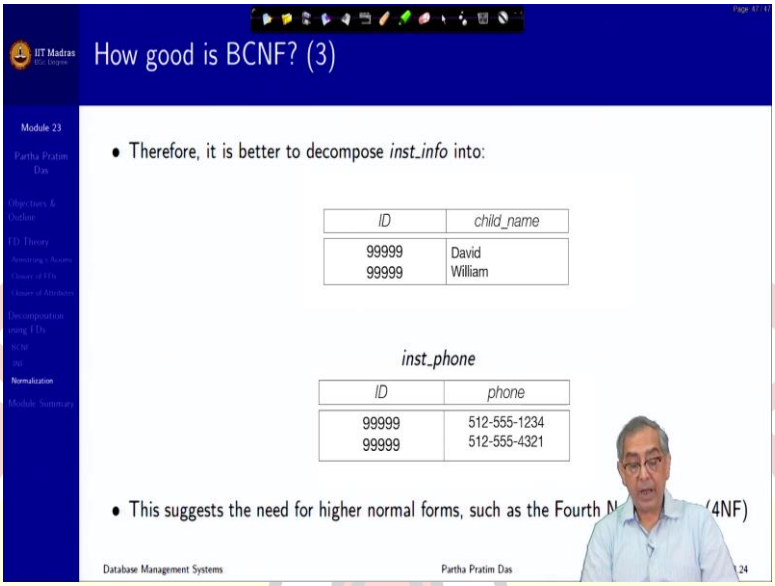
(Refer Slide Time: 33:13)



So, there is no non-trivial functional dependency here, nothing determines anything. So, the relation is in BCNF. Now, there could be insertion anomalies, because if we have to add a phone number, then we need to add two tuples because the instructors has two kids,

something even conceptually funny, or if you need to do an update, you need to do it twice and so on, bad situation.

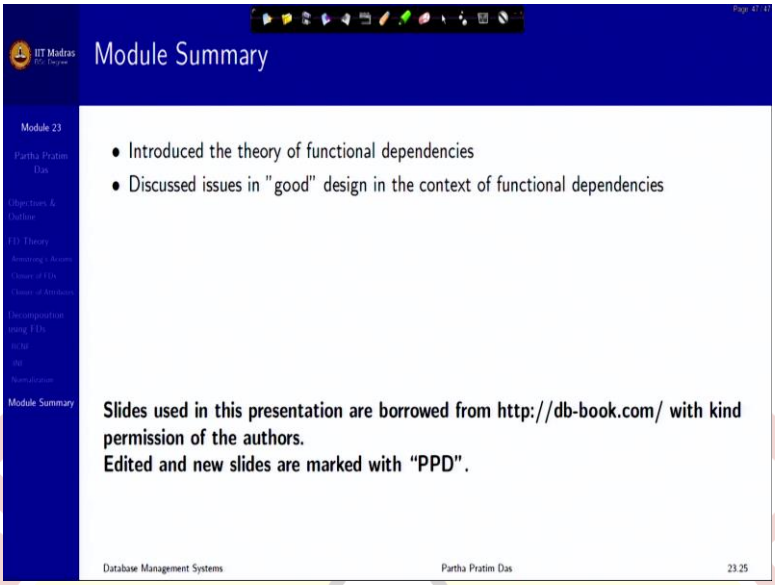
(Refer Slide Time: 33:46)



So, we need to decompose this and functional dependency is not telling us to decompose, mind you, the relation is in BCNF because you do not have a non-trivial dependency. But if you decompose, then your problem gets easier, because you can have now multiple children in one table, multiple phone numbers in the other table and you will be pretty good in terms of the normalised representation, the anomalies will go away.

And so, this leads to, since this is already in BCNF, actually it in three NF, so three NF is not going to solve this problem. So, this needs the, leads to even higher orders of normal forms, typically a fourth normal form of which we will talk later.

(Refer Slide Time: 34:49)



So, in this module you have introduced and expanded on the theory of functional dependencies and talked about the basic normalization forms BCNF and three NF of which we will have to discuss more and discuss the good design in the concept, in the context of functional dependencies. Thank you very much for your attention. This brings us to the end of this module. See you in the next one.